



通过示例学Java基础编程技巧

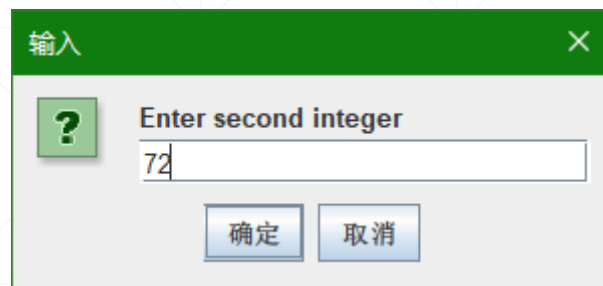
北京理工大学计算机学院
金旭亮

示例：两数相加

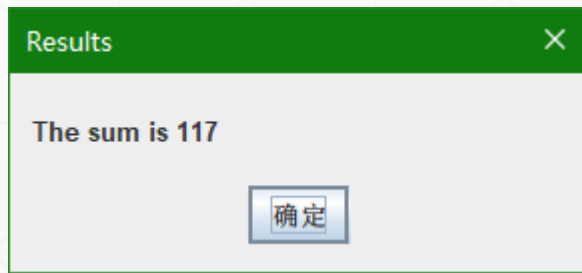
程序使用两个输入框输入数据，用一个消息框显示计算结果：



A Java Swing dialog box titled "输入" (Input) with a green header bar and a close button (X). It contains a green question mark icon, the text "Enter first integer", a text field with the value "45", and two buttons at the bottom: "确定" (OK) and "取消" (Cancel).



A Java Swing dialog box titled "输入" (Input) with a green header bar and a close button (X). It contains a green question mark icon, the text "Enter second integer", a text field with the value "72", and two buttons at the bottom: "确定" (OK) and "取消" (Cancel).



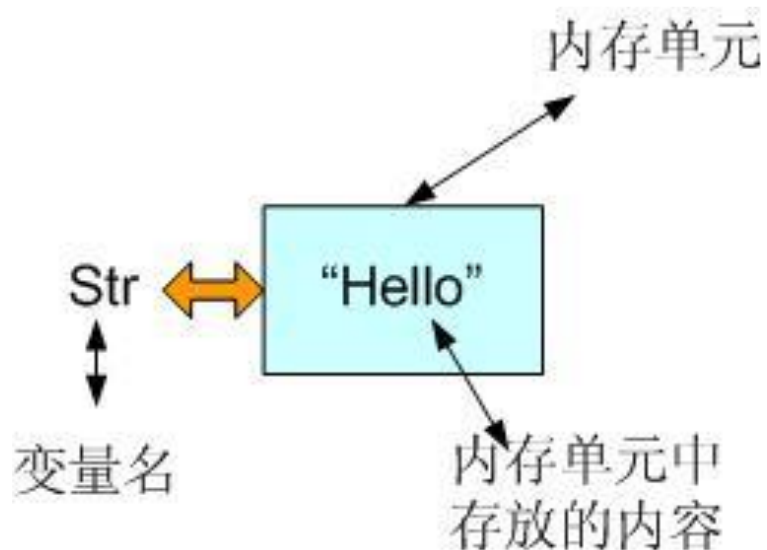
A Java Swing dialog box titled "Results" with a green header bar and a close button (X). It displays the text "The sum is 117" and has a single "确定" (OK) button at the bottom.

源文件：Addition.java

复习：Java中变量的内存模型

变量可看成是某内存单元（区域）的名字：

```
String str = "Hello" ;
```



定义变量时，需要指明变量的类型，在本例中，变量`str`的类型是`String`（即字符串）类型。

在Java 10中，可以使用`var`关键字定义变量，而由编译器自动依据变量初始值确定变量类型：

```
var str = "Hello" ;
```

变量与内存

每个变量都有一个名字、数据类型、所占内存单元数和一个值：

- **名字：** 对应于内存中的位置
- **数据类型：** 决定了它所占用的内存单元数量
- **值：** 表示变量所占用的内存单元中所保存的数据。

变量的读写： 当新值被赋给变量时，老值将被取代，**仅从内存中读数据不会破坏数据。**

变量的作用域

每个变量都有一个“有效”的区域（称为“**作用域**”），出了这个区域，变量将不再有效。

```
public class WhichIsUsed {  
    private static int value = 1;  
  
    public static void main(String[] args) {  
        int value = 2;  
        System.out.println(value);  
    }  
}
```

WhichIsUsed.java

请看左边的示例代码，输出结果是什么？
由此你能总结出什么样的结论？

课后练习：

Java变量遵循“**同名变量的屏蔽原则**”，
请课后阅读相关资料弄清楚相关知识，
然后自己编写一些测试代码，就像本示例一样，有意识地在不同地方定义一些同名变量，看看输出的到底是哪个值。

变量的使用准则



在实际开发中，一般使用变量来存储用户在程序运行时输入的数据，变量名字最好能反映出它要保存的数据的含义，比如，“`int age = 18;`”，这个变量一看就知道它保存的数据代表一个人的年龄。



变量在使用前应保证它有确切的值。

下面简要介绍一下如何在Java应用程序中读入数据.....

在运行时读取用户输入 (1)

方法一：使用JOptionPane类的showInputDialog方法。

```
String firstNumber =  
    JOptionPane.showInputDialog("Enter first integer :" );
```



注意一下当消息框关闭时，如果用户有输入，其结果是一个字符串。



试一试：

如果用户点击“取消”按钮，firstNumber会得到一个什么值呢？

在运行时读取用户输入 (2)

方法二：使用Scanner类（JDK 5.0 以上）

```
//System.in代表系统标准输出流，通常这个就是指键盘  
Scanner in = new Scanner(System.in);
```

```
//获取第一个输入
```

```
System.out.print("你的姓名? ");
```

```
//将用户输入视为字符串
```

```
String name = in.nextLine();
```

```
// 获取第二个输入
```

```
System.out.print("你多大了? ");
```

```
//将用户输入视为整数
```

```
int age = in.nextInt();
```

```
//输出结果
```

```
System.out.println( name + "你好! 明年, 你就 "  
                    + (age + 1) + "岁了。");
```

InputTest.java

Scanner类在java.util包中，使用前要先导入它：

```
Imports java.util.*;
```

Scanner类有 nextInt ， nextDouble 等方法，可以直接地将输入转换为数值。



```
Run: InputTest x  
"C:\Program Files\Java\jdk-15\bin\java.exe"  
你的姓名? 张三  
你多大了? 23  
张三你好! 明年, 你就 24岁了。
```


变量间的类型转换

→ 自动类型转换是安全的：

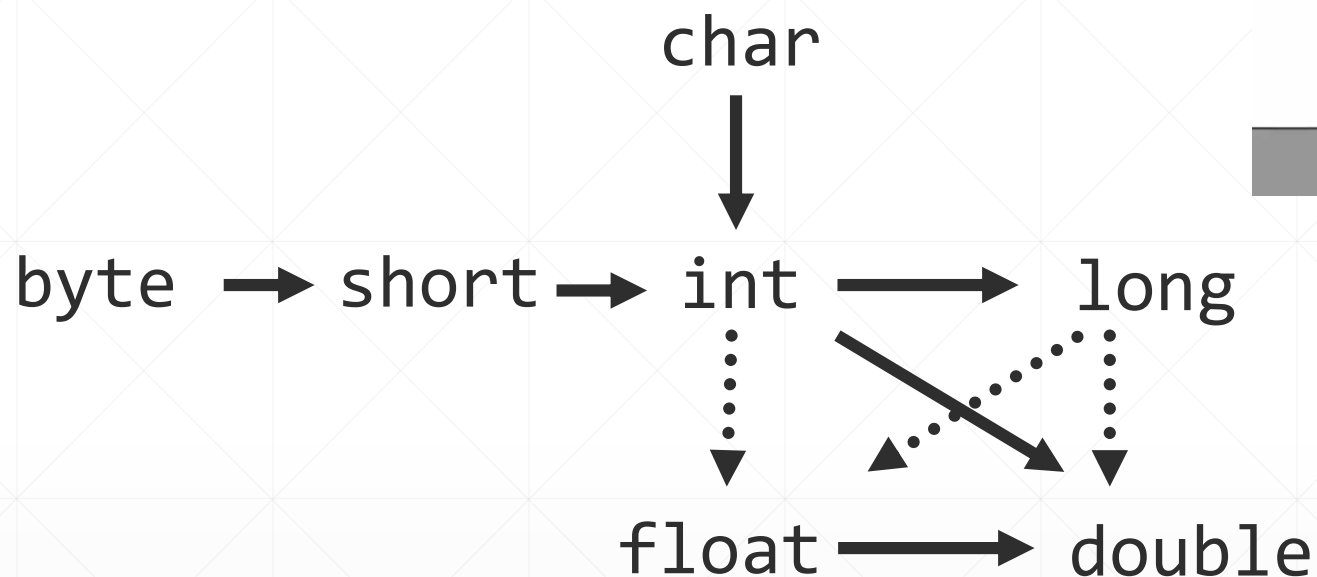
```
int intValue = 100;  
long longValue = intValue;  
//两个数值大小还一样吗?  
System.out.println(intValue == longValue); //true
```

→ 强制类型转换时可能会引起信息的损失：

```
double doubleValue = 1234567890987654321d;  
float floatValue = (float) doubleValue;  
//floatValue能表达的数值范围有限，并不能精确地容纳  
//各种double类型的数值  
System.out.println(floatValue); //1.23456794E18  
//两个数值大小还一样吗?  
System.out.println(doubleValue == floatValue); //false
```

Java使用一对括号实现
变量类型的强制转换

Java中的类型转换



—————> 无精度损失
.....> 有精度损失

看着这个图，再查查Java中每个数据类型所占的位数和表示数值的范围，你能得出什么结论？

定义数值型变量时隐式指定数据类型

```
var myInt = 5;  
var myFloat = 5f;  
var myDouble = 5d;  
var myLong = 5L;
```

如果你使用`var`来定义变量，那么，通过在数值常量后面添加“后缀”，编译器能正确地推断出每个变量的正确类型。

另一种数据类型转换方法

除了使用强制类型转换方式，我们还可以通过原始类型的**包装类**完成类型转换：

```
double someDoubleValue = 156.5d;  
//创建double对应的包装类型  
Double doubleObj = Double.valueOf(someDoubleValue);  
//Double包装类型提供了很多种方法转换为其他类型的数值  
byte myByteValue = doubleObj.byteValue();  
int myIntValue = doubleObj.intValue();  
float myFloatValue = doubleObj.floatValue();  
String myString = doubleObj.toString();
```

本方法的适用场景：

同一个数据需要转换为多种类型，并且这一数据需要比较长期的使用。

多数情况下，推荐直接使用强制类型转换的方式。



将对应原始数据类型的名字首字母大写，就得到了对应的包装类型。包装类型是“**引用类型**”，所以，它拥有N多的方法，集成了诸多相关的功能。

动手实验



请运行以下代码 (TestDouble.java) :

```
public class TestDouble {  
  
    public static void main(String[] args) {  
        System.out.println("0.05 + 0.01 = " + (0.05 + 0.01));  
        System.out.println("1.0 - 0.42 = " + (1.0 - 0.42));  
        System.out.println("4.015 * 100 = " + (4.015 * 100));  
        System.out.println("123.3 / 100 = " + (123.3 / 100));  
    }  
}
```

你看到了什么样的输出，感到意外吗？

下页PPT有答案，不要提前偷看！

示例分析

出现灵异现象:

```
Run: TestDouble x
"C:\Program Files\Java\jdk-15\bin\java.exe"
0.05 + 0.01 = 0.060000000000000005
1.0 - 0.42 = 0.58000000000000001
4.015 * 100 = 401.49999999999994
123.3 / 100 = 1.2329999999999999

Process finished with exit code 0
```

这些数都是
什么鬼?

结论:

使用double类型的数值进行计算, 其结果是近似的。

你知道原因吗？

为什么double类型的数值进行运算得不到“数学上精确”的结果？



请通过互联网查找资料，或者是阅读相关计算机教材，解决这个问题。



这个问题，与浮点数在计算机内部的表示方法有关系。可以使用这个线索在搜索引擎中查找相关资料。

怎样处理精度损失？

解决方法—使用BigDecimal类！

```
public class TestBigDecimal
{
    public static void main(String[] args)
    {
        BigDecimal f1 = new BigDecimal( val: "0.05");
        BigDecimal f2 =new BigDecimal( val: "0.01");
        System.out.println("使用String作为BigDecimal构造器参数的计算结果: ");
        System.out.println("0.05 + 0.01 = " + f1.add(f2));
        System.out.println("0.05 - 0.01 = " + f1.subtract(f2));
        System.out.println("0.05 * 0.01 = " + f1.multiply(f2));
        System.out.println("0.05 / 0.01 = " + f1.divide(f2));

        System.out.println("使用double作为BigDecimal构造器参数的计算结果: ");
        BigDecimal f3 = new BigDecimal( val: 0.05);
        System.out.println("0.05 + 0.01 = " + f3.add(f2));
        System.out.println("0.05 - 0.01 = " + f3.subtract(f2));
        System.out.println("0.05 * 0.01 = " + f3.multiply(f2));
        System.out.println("0.05 / 0.01 = " + f3.divide(f2));
    }
}
```

TestBigDecimal.java

面对示例运行结果，陷入沉思.....



```
Run: TestBigDecimal x
"C:\Program Files\Java\jdk-15\bin\java.exe" "-javaagent"
使用String作为BigDecimal构造器参数的计算结果:
0.05 + 0.01 = 0.06
0.05 - 0.01 = 0.04
0.05 * 0.01 = 0.0005
0.05 / 0.01 = 5
使用double作为BigDecimal构造器参数的计算结果:
0.05 + 0.01 = 0.060000000000000000277555756156289135105907917022705078125
0.05 - 0.01 = 0.040000000000000000277555756156289135105907917022705078125
0.05 * 0.01 = 0.00050000000000000000277555756156289135105907917022705078125
0.05 / 0.01 = 5.0000000000000000277555756156289135105907917022705078125

Process finished with exit code 0
```

注意：在构建BigDecimal对象时应使用字符串而不是double数值，否则，仍有可能引发计算精度问题。（**为什么会这样呢？**）

字符串转为数字的基本方法

```
int number = Integer.parseInt(numberString );
```

- Integer.parseInt方法将 String 转为数字 (int)
- 类Integer 属于包 java.lang, 它“封装”了一个int类型的整数, 因此, 它是原始数据类型int的“**包装类**”。

Java中的包装类 (Primitive Wrapper Classes)

Java中有7种原始数据类型，都对应有相应的包装类，这些类型的用法都是类似的，掌握一个，其它的就基本上都会用了。

Data Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
boolean	Boolean
char	Character

字符串联接操作

使用运算符 “+” 连接字面量 "The sum is" 和计算结果 sum

```
JOptionPane.showMessageDialog(  
    null, "The sum is " + sum, "Results",  
    JOptionPane.PLAIN_MESSAGE );
```



将String和其它数据类型“相加”，结果是一个新的String。

动手动脑

以下代码的输出结果是什么？

```
int X = 100;  
int Y = 200;  
System.out.println("X+Y="+X+Y);  
System.out.println(X+Y+"=X+Y");
```

为什么会有这样的输出结果？



课后作业



编写一个程序，用户输入两个数，求出其加减乘除，并用消息框显示计算结果

学以致用



在网上时，我们经常会看到以下这种对话框，要用户输入一个验证码。

请登录

登录名: 用户名/邮箱

密码: 忘记用户名/密码?

验证码: 825

登录

快速注册

如果要由你写一个自动生成验证码的程序，你能完成这个任务吗？

参考答案 (RandomStr.java)：使用类型转换生成6位验证字符串，程序每次运行时，都会生成不同的字符串

下一讲，介绍Java编程中的各种语句.....